WEEK 8: SUPERMARKET MANAGEMENT SYSTEM Logeshwari D (E25AI121)

1. Scenario

The Supermarket Management System stores customer, product, sales, and payment information in separate tables. The JOIN operation combines related tables to retrieve customer purchase details, product sales, and payment information.

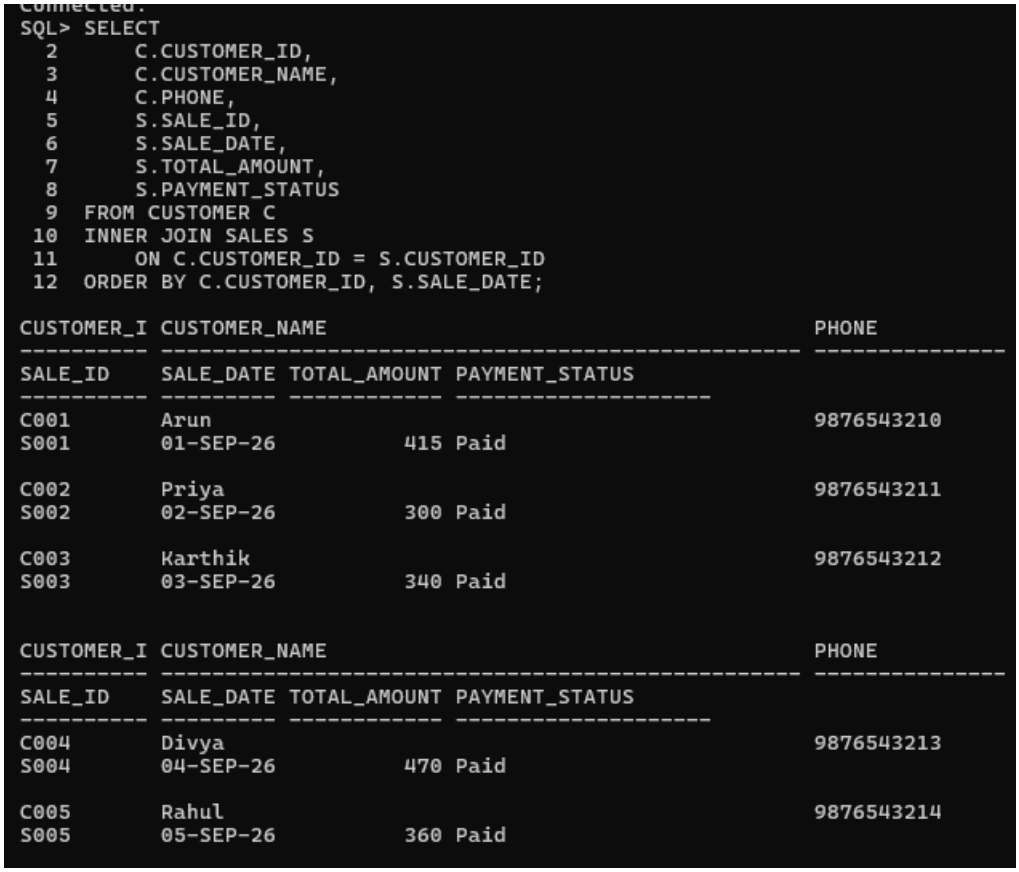
The system helps generate complete order details, customer purchase history, sales reports, and payment analysis.

2. Tables used

Table	**Purpose**
CUSTOMER	Stores customer details
SALES	Stores sale date, total amount, and payment status
SALE_DETAILS	Stores products, quantities, and prices for each sale
PRODUCT	Stores product details
PAYMENT	Stores payment mode, transaction amount, and payment status

3. Implement INNER JOIN

Task 1: Display customer details along with their orders



```
Connected.
SQL> SELECT
  2      C.CUSTOMER_ID,
  3      C.CUSTOMER_NAME,
  4      C.PHONE,
  5      S.SALE_ID,
  6      S.SALE_DATE,
  7      S.TOTAL_AMOUNT,
  8      S.PAYMENT_STATUS
  9  FROM CUSTOMER C
 10  INNER JOIN SALES S
 11      ON C.CUSTOMER_ID = S.CUSTOMER_ID
 12  ORDER BY C.CUSTOMER_ID, S.SALE_DATE;

CUSTOMER_I CUSTOMER_NAME                                      PHONE
---------- -------------------------------------------------- ---------------
SALE_ID    SALE_DATE TOTAL_AMOUNT PAYMENT_STATUS
---------- --------- ------------ --------------------
C001       Arun                                               9876543210
S001       01-SEP-26          415 Paid

C002       Priya                                              9876543211
S002       02-SEP-26          300 Paid

C003       Karthik                                            9876543212
S003       03-SEP-26          340 Paid


CUSTOMER_I CUSTOMER_NAME                                      PHONE
---------- -------------------------------------------------- ---------------
SALE_ID    SALE_DATE TOTAL_AMOUNT PAYMENT_STATUS
---------- --------- ------------ --------------------
C004       Divya                                              9876543213
S004       04-SEP-26          470 Paid

C005       Rahul                                              9876543214
S005       05-SEP-26          360 Paid
```

Task 2: Display order details with payment information

```
SQL> SELECT
  2      S.SALE_ID,
  3      S.SALE_DATE,
  4      S.TOTAL_AMOUNT,
  5      P.PAYMENT_ID,
  6      P.PAYMENT_DATE,
  7      P.PAYMENT_MODE,
  8      P.TRANSACTION_AMOUNT,
  9      P.PAYMENT_STATUS
 10  FROM SALES S
 11  INNER JOIN PAYMENT P
 12      ON S.SALE_ID = P.SALE_ID
 13  ORDER BY S.SALE_ID;

SALE_ID    SALE_DATE TOTAL_AMOUNT PAYMENT_ID PAYMENT_D PAYMENT_MODE
---------- --------- ------------ ---------- --------- --------------------
TRANSACTION_AMOUNT PAYMENT_STATUS
------------------ --------------------
S001       01-SEP-26          415 PAY001     01-SEP-26 UPI
               415 SUCCESS

S002       02-SEP-26          300 PAY002     02-SEP-26 CASH
               300 SUCCESS

S003       03-SEP-26          340 PAY003     03-SEP-26 CARD
               340 SUCCESS


SALE_ID    SALE_DATE TOTAL_AMOUNT PAYMENT_ID PAYMENT_D PAYMENT_MODE
---------- --------- ------------ ---------- --------- --------------------
TRANSACTION_AMOUNT PAYMENT_STATUS
------------------ --------------------
S004       04-SEP-26          470 PAY004     04-SEP-26 UPI
               470 SUCCESS

S005       05-SEP-26          360 PAY005     05-SEP-26 CASH
               360 SUCCESS
```

Task 3: Retrieve products purchased by customers

```
SQL> SELECT
  2      C.CUSTOMER_NAME,
  3      S.SALE_ID,
  4      P.PRODUCT_ID,
  5      P.PRODUCT_NAME,
  6      SD.QUANTITY,
  7      SD.PRICE
  8  FROM CUSTOMER C
  9  INNER JOIN SALES S
 10      ON C.CUSTOMER_ID = S.CUSTOMER_ID
 11  INNER JOIN SALE_DETAILS SD
 12      ON S.SALE_ID = SD.SALE_ID
 13  INNER JOIN PRODUCT P
 14      ON SD.PRODUCT_ID = P.PRODUCT_ID
 15  ORDER BY C.CUSTOMER_NAME, S.SALE_ID;

CUSTOMER_NAME                                      SALE_ID    PRODUCT_ID
-------------------------------------------------- ---------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE
---------- ----------
Arun                                               S001       P001
Rice 5kg
         1        350

Arun                                               S001       P002
Wheat Flour 1kg
         1         65

CUSTOMER_NAME                                      SALE_ID    PRODUCT_ID
-------------------------------------------------- ---------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE
---------- ----------

Divya                                              S004       P009
Chocolate
         1         80

Divya                                              S004       P010
Shampoo

CUSTOMER_NAME                                      SALE_ID    PRODUCT_ID
-------------------------------------------------- ---------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE
---------- ----------
         1        220

Divya                                              S004       P005
Green Tea
         1        180

Karthik                                            S003       P007
```

```
CUSTOMER_NAME                                      SALE_ID    PRODUCT_ID
-------------------------------------------------- ---------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE
---------- ----------
Potato Chips
         1         40

Karthik                                            S003       P003
Sugar 1kg
         2         55


CUSTOMER_NAME                                      SALE_ID    PRODUCT_ID
-------------------------------------------------- ---------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE
---------- ----------
Karthik                                            S003       P008
Biscuits
         1         30

Priya                                              S002       P005
Green Tea
         1        180

CUSTOMER_NAME                                      SALE_ID    PRODUCT_ID
-------------------------------------------------- ---------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE
---------- ----------

Priya                                              S002       P004
Orange Juice
         1        120

Rahul                                              S005       P008
Biscuits

CUSTOMER_NAME                                      SALE_ID    PRODUCT_ID
-------------------------------------------------- ---------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE
---------- ----------
         1         30

Rahul                                              S005       P015
Laundry Detergent
         1        280

Rahul                                              S005       P006

CUSTOMER_NAME                                      SALE_ID    PRODUCT_ID
-------------------------------------------------- ---------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE
---------- ----------
Mineral Water
         1         25


13 rows selected.
```

4. Implement LEFT JOIN

Task 1: Display all customers, including customers who have not placed orders

```
SQL> SELECT
  2      C.CUSTOMER_ID,
  3      C.CUSTOMER_NAME,
  4      S.SALE_ID,
  5      S.SALE_DATE,
  6      S.TOTAL_AMOUNT
  7  FROM CUSTOMER C
  8  LEFT JOIN SALES S
  9      ON C.CUSTOMER_ID = S.CUSTOMER_ID
 10  ORDER BY C.CUSTOMER_ID;

CUSTOMER_I CUSTOMER_NAME                                      SALE_ID
---------- -------------------------------------------------- ----------
SALE_DATE TOTAL_AMOUNT
--------- ------------
C001       Arun                                               S001
01-SEP-26          415

C002       Priya                                              S002
02-SEP-26          300

C003       Karthik                                            S003
03-SEP-26          340


CUSTOMER_I CUSTOMER_NAME                                      SALE_ID
---------- -------------------------------------------------- ----------
SALE_DATE TOTAL_AMOUNT
--------- ------------
C004       Divya                                              S004
04-SEP-26          470

C005       Rahul                                              S005
05-SEP-26          360
```

Task 2: Find customers without any purchases

```
SQL> SELECT
  2      C.CUSTOMER_ID,
  3      C.CUSTOMER_NAME,
  4      C.PHONE,
  5      C.EMAIL
  6  FROM CUSTOMER C
  7  LEFT JOIN SALES S
  8      ON C.CUSTOMER_ID = S.CUSTOMER_ID
  9  WHERE S.SALE_ID IS NULL;

no rows selected
```

Task 3: Display all products, including products with no sales

```
SQL> SELECT
  2      P.PRODUCT_ID,
  3      P.PRODUCT_NAME,
  4      P.PRICE,
  5      SD.SALE_ID,
  6      SD.QUANTITY
  7  FROM PRODUCT P
  8  LEFT JOIN SALE_DETAILS SD
  9      ON P.PRODUCT_ID = SD.PRODUCT_ID
 10  ORDER BY P.PRODUCT_ID;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
     PRICE SALE_ID      QUANTITY
---------- ---------- ----------
P001
Rice 5kg
       350 S001               1

P002
Wheat Flour 1kg
        65 S001               1

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
     PRICE SALE_ID      QUANTITY
---------- ---------- ----------

P003
Sugar 1kg
        55 S003               2

P004
Orange Juice

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
     PRICE SALE_ID      QUANTITY
---------- ---------- ----------
       120 S002               1

P005
Green Tea
       180 S004               1

P005

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
     PRICE SALE_ID      QUANTITY
---------- ---------- ----------
Green Tea
       180 S002               1

P006
Mineral Water
        25 S005               1
```

```
PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
     PRICE SALE_ID      QUANTITY
---------- ---------- ----------
P007
Potato Chips
        40 S003                1

P008
Biscuits
        30 S005                1

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
     PRICE SALE_ID      QUANTITY
---------- ---------- ----------

P008
Biscuits
        30 S003                1

P009
Chocolate

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
     PRICE SALE_ID      QUANTITY
---------- ---------- ----------
        80 S004                1

P010
Shampoo
       220 S004                1

P011

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
     PRICE SALE_ID      QUANTITY
---------- ---------- ----------
Bath Soap
        45

P012
Toothpaste
       110
```

```
PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
     PRICE SALE_ID      QUANTITY
---------- ---------- ----------
P013
Dishwash Liquid
       150

P014
Floor Cleaner
       190

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
     PRICE SALE_ID      QUANTITY
---------- ---------- ----------

P015
Laundry Detergent
       280 S005                1


17 rows selected.
```

5. Implement RIGHT JOIN

Task 1: Display all orders with customer information

```
SQL> SELECT
  2      S.SALE_ID,
  3      S.SALE_DATE,
  4      S.TOTAL_AMOUNT,
  5      S.PAYMENT_STATUS,
  6      C.CUSTOMER_ID,
  7      C.CUSTOMER_NAME
  8  FROM CUSTOMER C
  9  RIGHT JOIN SALES S
 10      ON C.CUSTOMER_ID = S.CUSTOMER_ID
 11  ORDER BY S.SALE_ID;

SALE_ID    SALE_DATE TOTAL_AMOUNT PAYMENT_STATUS       CUSTOMER_I
---------- --------- ------------ -------------------- ----------
CUSTOMER_NAME
--------------------------------------------------
S001       01-SEP-26          415 Paid                 C001
Arun

S002       02-SEP-26          300 Paid                 C002
Priya

S003       03-SEP-26          340 Paid                 C003
Karthik


SALE_ID    SALE_DATE TOTAL_AMOUNT PAYMENT_STATUS       CUSTOMER_I
---------- --------- ------------ -------------------- ----------
CUSTOMER_NAME
--------------------------------------------------
S004       04-SEP-26          470 Paid                 C004
Divya

S005       05-SEP-26          360 Paid                 C005
Rahul
```

Task 2: Find orders where customer details are missing

```
SQL> SELECT
  2      S.SALE_ID,
  3      S.SALE_DATE,
  4      S.TOTAL_AMOUNT,
  5      S.CUSTOMER_ID
  6  FROM CUSTOMER C
  7  RIGHT JOIN SALES S
  8      ON C.CUSTOMER_ID = S.CUSTOMER_ID
  9  WHERE C.CUSTOMER_ID IS NULL;

no rows selected
```

Task 3: Display all payment records with order details

```
SQL> SELECT
  2      P.PAYMENT_ID,
  3      P.SALE_ID,
  4      P.PAYMENT_DATE,
  5      P.PAYMENT_MODE,
  6      P.TRANSACTION_AMOUNT,
  7      P.PAYMENT_STATUS,
  8      S.SALE_DATE,
  9      S.TOTAL_AMOUNT
 10  FROM SALES S
 11  RIGHT JOIN PAYMENT P
 12      ON S.SALE_ID = P.SALE_ID
 13  ORDER BY P.PAYMENT_ID;

PAYMENT_ID SALE_ID    PAYMENT_D PAYMENT_MODE         TRANSACTION_AMOUNT
---------- ---------- --------- -------------------- ------------------
PAYMENT_STATUS       SALE_DATE TOTAL_AMOUNT
-------------------- --------- ------------
PAY001     S001       01-SEP-26 UPI                                415
SUCCESS              01-SEP-26          415

PAY002     S002       02-SEP-26 CASH                               300
SUCCESS              02-SEP-26          300

PAY003     S003       03-SEP-26 CARD                               340
SUCCESS              03-SEP-26          340


PAYMENT_ID SALE_ID    PAYMENT_D PAYMENT_MODE         TRANSACTION_AMOUNT
---------- ---------- --------- -------------------- ------------------
PAYMENT_STATUS       SALE_DATE TOTAL_AMOUNT
-------------------- --------- ------------
PAY004     S004       04-SEP-26 UPI                                470
SUCCESS              04-SEP-26          470

PAY005     S005       05-SEP-26 CASH                               360
SUCCESS              05-SEP-26          360
```

6. Retrieve complete order details

```
SQL> SELECT
  2      C.CUSTOMER_NAME,
  3      P.PRODUCT_NAME,
  4      SD.QUANTITY,
  5      S.SALE_DATE AS ORDER_DATE,
  6      S.TOTAL_AMOUNT,
  7      PM.PAYMENT_STATUS
  8  FROM CUSTOMER C
  9  INNER JOIN SALES S
 10      ON C.CUSTOMER_ID = S.CUSTOMER_ID
 11  INNER JOIN SALE_DETAILS SD
 12      ON S.SALE_ID = SD.SALE_ID
 13  INNER JOIN PRODUCT P
 14      ON SD.PRODUCT_ID = P.PRODUCT_ID
 15  LEFT JOIN PAYMENT PM
 16      ON S.SALE_ID = PM.SALE_ID
 17  ORDER BY S.SALE_DATE DESC;

CUSTOMER_NAME
--------------------------------------------------
PRODUCT_NAME
--------------------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT PAYMENT_STATUS
---------- --------- ------------ --------------------
Rahul
Mineral Water
         1 05-SEP-26          360 SUCCESS

Rahul
Biscuits
         1 05-SEP-26          360 SUCCESS

CUSTOMER_NAME
--------------------------------------------------
PRODUCT_NAME
--------------------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT PAYMENT_STATUS
---------- --------- ------------ --------------------

Rahul
Laundry Detergent
         1 05-SEP-26          360 SUCCESS

Divya
Shampoo

CUSTOMER_NAME
--------------------------------------------------
PRODUCT_NAME
--------------------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT PAYMENT_STATUS
---------- --------- ------------ --------------------
         1 04-SEP-26          470 SUCCESS

Divya
Green Tea
         1 04-SEP-26          470 SUCCESS

Divya
```

```
CUSTOMER_NAME
--------------------------------------------------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT PAYMENT_STATUS
---------- --------- ------------ --------------------
Chocolate
         1 04-SEP-26          470 SUCCESS

Karthik
Potato Chips
         1 03-SEP-26          340 SUCCESS


CUSTOMER_NAME
--------------------------------------------------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT PAYMENT_STATUS
---------- --------- ------------ --------------------
Karthik
Sugar 1kg
         2 03-SEP-26          340 SUCCESS

Karthik
Biscuits
         1 03-SEP-26          340 SUCCESS

CUSTOMER_NAME
--------------------------------------------------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT PAYMENT_STATUS
---------- --------- ------------ --------------------

Priya
Orange Juice
         1 02-SEP-26          300 SUCCESS

Priya
Green Tea

CUSTOMER_NAME
--------------------------------------------------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT PAYMENT_STATUS
---------- --------- ------------ --------------------
         1 02-SEP-26          300 SUCCESS

Arun
Wheat Flour 1kg
         1 01-SEP-26          415 SUCCESS

Arun

CUSTOMER_NAME
--------------------------------------------------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT PAYMENT_STATUS
---------- --------- ------------ --------------------
Rice 5kg
         1 01-SEP-26          415 SUCCESS


13 rows selected.
```

7. Display customer purchase history

Task 1: Display all products purchased by a customer

```
SQL> SELECT
  2      C.CUSTOMER_NAME,
  3      S.SALE_ID,
  4      S.SALE_DATE,
  5      P.PRODUCT_NAME,
  6      SD.QUANTITY,
  7      SD.PRICE
  8  FROM CUSTOMER C
  9  INNER JOIN SALES S
 10      ON C.CUSTOMER_ID = S.CUSTOMER_ID
 11  INNER JOIN SALE_DETAILS SD
 12      ON S.SALE_ID = SD.SALE_ID
 13  INNER JOIN PRODUCT P
 14      ON SD.PRODUCT_ID = P.PRODUCT_ID
 15  WHERE C.CUSTOMER_ID = 'C001'
 16  ORDER BY S.SALE_DATE DESC;

CUSTOMER_NAME                                      SALE_ID    SALE_DATE
-------------------------------------------------- ---------- ---------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE
---------- ----------
Arun                                               S001       01-SEP-26
Wheat Flour 1kg
         1         65

Arun                                               S001       01-SEP-26
Rice 5kg
         1        350

CUSTOMER_NAME                                      SALE_ID    SALE_DATE
-------------------------------------------------- ---------- ---------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE
---------- ----------
```

Task 2: Calculate the total amount spent by each customer

```
SQL> SELECT
  2      C.CUSTOMER_ID,
  3      C.CUSTOMER_NAME,
  4      NVL(SUM(S.TOTAL_AMOUNT), 0) AS TOTAL_AMOUNT_SPENT
  5  FROM CUSTOMER C
  6  LEFT JOIN SALES S
  7      ON C.CUSTOMER_ID = S.CUSTOMER_ID
  8  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  9  ORDER BY TOTAL_AMOUNT_SPENT DESC;

CUSTOMER_I CUSTOMER_NAME                                      TOTAL_AMOUNT_SPENT
---------- -------------------------------------------------- ------------------
C004       Divya                                                             470
C001       Arun                                                              415
C005       Rahul                                                             360
C003       Karthik                                                           340
C002       Priya                                                             300
```

Task 3: Find the number of orders placed by each customer

```
SQL> SELECT
  2      C.CUSTOMER_ID,
  3      C.CUSTOMER_NAME,
  4      COUNT(S.SALE_ID) AS NUMBER_OF_ORDERS
  5  FROM CUSTOMER C
  6  LEFT JOIN SALES S
  7      ON C.CUSTOMER_ID = S.CUSTOMER_ID
  8  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  9  ORDER BY NUMBER_OF_ORDERS DESC;

CUSTOMER_I CUSTOMER_NAME                                      NUMBER_OF_ORDERS
---------- -------------------------------------------------- ----------------
C004       Divya                                                             1
C003       Karthik                                                           1
C002       Priya                                                             1
C001       Arun                                                              1
C005       Rahul                                                             1
```

Task 4: Display the latest purchase details of each customer

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      CUSTOMER_NAME,
  4      SALE_ID,
  5      SALE_DATE,
  6      TOTAL_AMOUNT,
  7      PAYMENT_STATUS
  8  FROM (
  9      SELECT
 10          C.CUSTOMER_ID,
 11          C.CUSTOMER_NAME,
 12          S.SALE_ID,
 13          S.SALE_DATE,
 14          S.TOTAL_AMOUNT,
 15          S.PAYMENT_STATUS,
 16          ROW_NUMBER() OVER (
 17              PARTITION BY C.CUSTOMER_ID
 18              ORDER BY S.SALE_DATE DESC NULLS LAST,
 19                       S.SALE_ID DESC
 20          ) AS RN
 21      FROM CUSTOMER C
 22      LEFT JOIN SALES S
 23          ON C.CUSTOMER_ID = S.CUSTOMER_ID
 24  )
 25  WHERE RN = 1
 26  ORDER BY CUSTOMER_ID;

CUSTOMER_I CUSTOMER_NAME                                      SALE_ID
---------- -------------------------------------------------- ----------
SALE_DATE TOTAL_AMOUNT PAYMENT_STATUS
--------- ------------ --------------------
C001       Arun                                               S001
01-SEP-26          415 Paid

C002       Priya                                              S002
02-SEP-26          300 Paid

C003       Karthik                                            S003
03-SEP-26          340 Paid


CUSTOMER_I CUSTOMER_NAME                                      SALE_ID
---------- -------------------------------------------------- ----------
SALE_DATE TOTAL_AMOUNT PAYMENT_STATUS
--------- ------------ --------------------
C004       Divya                                              S004
04-SEP-26          470 Paid

C005       Rahul                                              S005
05-SEP-26          360 Paid
```

8. Generate multi-table business reports

Required details:

1)Customer name

2)Order ID

3)Order date

4)Order status

```
SQL> SELECT
  2      C.CUSTOMER_NAME,
  3      S.SALE_ID AS ORDER_ID,
  4      S.SALE_DATE AS ORDER_DATE,
  5      S.PAYMENT_STATUS AS ORDER_STATUS
  6  FROM CUSTOMER C
  7  INNER JOIN SALES S
  8      ON C.CUSTOMER_ID = S.CUSTOMER_ID
  9  ORDER BY S.SALE_DATE DESC;

CUSTOMER_NAME                                      ORDER_ID   ORDER_DAT
-------------------------------------------------- ---------- ---------
ORDER_STATUS
--------------------
Rahul                                              S005       05-SEP-26
Paid

Divya                                              S004       04-SEP-26
Paid

Karthik                                            S003       03-SEP-26
Paid


CUSTOMER_NAME                                      ORDER_ID   ORDER_DAT
-------------------------------------------------- ---------- ---------
ORDER_STATUS
--------------------
Priya                                              S002       02-SEP-26
Paid

Arun                                               S001       01-SEP-26
Paid
```

Report 2: Sales Report

Required details:

1)Product name

2)Quantity sold

3)Total revenue

```
SQL> SELECT
  2      P.PRODUCT_NAME,
  3      SUM(SD.QUANTITY) AS QUANTITY_SOLD,
  4      SUM(SD.QUANTITY * SD.PRICE) AS TOTAL_REVENUE
  5  FROM PRODUCT P
  6  INNER JOIN SALE_DETAILS SD
  7      ON P.PRODUCT_ID = SD.PRODUCT_ID
  8  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  9  ORDER BY TOTAL_REVENUE DESC;

PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
Green Tea
            2           360

Rice 5kg
            1           350

Laundry Detergent
            1           280


PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
Shampoo
            1           220

Orange Juice
            1           120

Sugar 1kg
            2           110


PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
Chocolate
            1            80

Wheat Flour 1kg
            1            65

Biscuits
            2            60


PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
Potato Chips
            1            40

Mineral Water
            1            25


11 rows selected.
```

Report 3: Payment Analysis Report

Required details:

1)Payment mode

2)Number of transactions

3)Successful payments.

```
SQL> SELECT
  2      PAYMENT_MODE,
  3      COUNT(*) AS NUMBER_OF_TRANSACTIONS,
  4      SUM(
  5          CASE
  6              WHEN PAYMENT_STATUS = 'Success' THEN 1
  7              ELSE 0
  8          END
  9      ) AS SUCCESSFUL_PAYMENTS
 10  FROM PAYMENT
 11  GROUP BY PAYMENT_MODE
 12  ORDER BY NUMBER_OF_TRANSACTIONS DESC;

PAYMENT_MODE         NUMBER_OF_TRANSACTIONS SUCCESSFUL_PAYMENTS
-------------------- ---------------------- ------------------
CASH                                     2                  0
UPI                                      2                  0
CARD                                     1                  0

SQL>
```

Report 4: Customer Purchase Analysis

Find

1)top purchasing customers

```
SQL> SELECT
  2      C.CUSTOMER_ID,
  3      C.CUSTOMER_NAME,
  4      NVL(SUM(SD.QUANTITY), 0) AS TOTAL_QUANTITY_PURCHASED
  5  FROM CUSTOMER C
  6  LEFT JOIN SALES S
  7      ON C.CUSTOMER_ID = S.CUSTOMER_ID
  8  LEFT JOIN SALE_DETAILS SD
  9      ON S.SALE_ID = SD.SALE_ID
 10  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
 11  ORDER BY TOTAL_QUANTITY_PURCHASED DESC;

CUSTOMER_I CUSTOMER_NAME
---------- --------------------------------------------------
TOTAL_QUANTITY_PURCHASED
------------------------
C003       Karthik
                      4

C004       Divya
                      3

C005       Rahul
                      3


CUSTOMER_I CUSTOMER_NAME
---------- --------------------------------------------------
TOTAL_QUANTITY_PURCHASED
------------------------
C001       Arun
                      2

C002       Priya
                      2
```

Find

2)customers with the maximum number of orders

```
SQL> WITH CUSTOMER_ORDERS AS (
  2      SELECT
  3          C.CUSTOMER_ID,
  4          C.CUSTOMER_NAME,
  5          COUNT(S.SALE_ID) AS NUMBER_OF_ORDERS
  6      FROM CUSTOMER C
  7      LEFT JOIN SALES S
  8          ON C.CUSTOMER_ID = S.CUSTOMER_ID
  9      GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
 10  )
 11  SELECT *
 12  FROM CUSTOMER_ORDERS
 13  WHERE NUMBER_OF_ORDERS = (
 14      SELECT MAX(NUMBER_OF_ORDERS)
 15      FROM CUSTOMER_ORDERS
 16  );

CUSTOMER_I CUSTOMER_NAME                                      NUMBER_OF_ORDERS
---------- -------------------------------------------------- ----------------
C004       Divya                                                             1
C003       Karthik                                                           1
C005       Rahul                                                             1
C001       Arun                                                              1
C002       Priya                                                             1
```

Find

3)customers with the highest spending

```
SQL> WITH CUSTOMER_SPENDING AS (
  2      SELECT
  3          C.CUSTOMER_ID,
  4          C.CUSTOMER_NAME,
  5          NVL(SUM(S.TOTAL_AMOUNT), 0) AS TOTAL_SPENDING
  6      FROM CUSTOMER C
  7      LEFT JOIN SALES S
  8          ON C.CUSTOMER_ID = S.CUSTOMER_ID
  9      GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
 10  )
 11  SELECT *
 12  FROM CUSTOMER_SPENDING
 13  WHERE TOTAL_SPENDING = (
 14      SELECT MAX(TOTAL_SPENDING)
 15      FROM CUSTOMER_SPENDING
 16  );

CUSTOMER_I CUSTOMER_NAME                                     TOTAL_SPENDING
---------- -------------------------------------------------- --------------
C004       Divya                                                         470
```

9. Conclusion

The Week 8 Supermarket Management System extends the previous weeks by combining customer, sales, product, and payment information using SQL JOIN operations.

The project demonstrates INNER JOIN, LEFT JOIN, and RIGHT JOIN to retrieve matching records, include records without matches, and identify missing relationships. It also generates complete order details, customer purchase history, sales reports, payment analysis, and customer purchase analysis.

Functions and clauses such as SUM(), COUNT(), GROUP BY, ORDER BY, NVL(), and ROW_NUMBER() help analyze purchasing activity and generate useful business reports.

Thus, the project demonstrates how database relationships and JOIN operations support effective supermarket management and decision-making.